Ignacio Agustín Duque Sandoval

AI Trainer · Evaluación de LLMs y agentes · RAG · Inferencia on-device

Argentina | Chile · Remoto · nachodusan@gmail.com · +56 9 2188 1312
dusan.codes · github.com/Nachodusan · linkedin.com/in/ignaciodusan

PERFIL

Un número que nadie auditó no es un resultado. Construyo sistemas con LLMs y el instrumento que los juzga —golden sets, verificación de trayectoria, medición de alucinación, benchmarks a recall igualado— y publico los dos, incluidas las veces en que el instrumento estaba mal. Siete proyectos con evidencia reproducible, de los cuales cuatro son herramientas de medición y los tres hallazgos que más enseñaron fueron resultados negativos: un benchmark inerte, una ruta de recuperación que nunca se dispara y tres iteraciones de fine-tuning descartadas con las métricas a la vista.

PROYECTOS · EVALUACIÓN Y MEDICIÓN

agent-evals — framework de evaluación de agentes, sin dependencias en el núcleo

`github.com/Nachodusan/agent-evals`

- Verificación de trayectoria: qué herramientas llamó el agente, en qué orden y de qué errores se recuperó. La respuesta final es sólo una de las comprobaciones.
- Un caso pasa **si pasaron todos los intentos**; lo intermedio se reporta como inestable en vez de redondearse a un porcentaje que no reproduce.
- Apuntado a mi propio sql-agent encontró tres cosas que una sola corrida no puede ver: un caso que “pasaba” era **una moneda al aire** (0/3, después 1/3), el agente reportaba SQL que nunca ejecutó, y un cuarto de las fallas del modelo débil eran crashes dentro de mi código.
- Python, sin dependencias en el core. 30 tests.

sql-agent — agente de lenguaje natural a SQL con el loop instrumentado `github.com/Nachodusan/sql-agent`

- Loop LangGraph con tool calling: inspecciona el esquema, escribe SQL, lo ejecuta y responde. Cada llamada queda registrada con latencia, tokens y costo, y la corrida completa serializa a JSON.
- **6/8 en el golden set con DeepSeek-V3 por US$ 0,0136**, y las dos fallas son SQL válido que devuelve números plausibles y equivocados. Por eso la corrección se mide contra queries de referencia y no contra la prosa, que se lee perfecta en los dos casos.
- La recuperación de errores que fui a demostrar **nunca se disparó**: 0 % medido en ocho preguntas y dos modelos, porque ambos consultan el esquema antes de escribir. El hallazgo quedó publicado en lugar del que buscaba.
- Python, LangGraph, SQLite sembrado con trampas (fan-out de joins, montos en centavos, pedidos anulados).

vector-db-bench — Qdrant vs. Chroma vs. pgvector a recall igualado `github.com/Nachodusan/vector-db-bench`

- 41.896 fragmentos embebidos con BGE-M3 (1024d), indexados con HNSW idéntico (*M*=16, *ef_construction*=200) en Apple M4. Comparar con los valores por defecto mide los defaults, así que la suite barre la perilla y mide recall contra fuerza bruta exacta.
- **La brecha crece con la precisión**: p95 de 1,4× a recall 0,90 y de 3,4× a 0,99. “Cuál es más rápida” no tiene respuesta hasta nombrar un recall objetivo.
- Chroma queda fuera de la tabla con el motivo escrito: acepta *ef_search*, lo persiste, lo reporta de vuelta y **es inerte** — resultados idénticos byte a byte para las 500 consultas en toda la barrida.
- La sección más larga del informe son las cinco formas en que el benchmark estuvo mal antes de estar bien, incluido un bug propio que inflaba la latencia de pgvector 2,4×. Tres de esas cinco hoy son guards automáticos.

tinyllm-ios — el throughput que un teléfono realmente sostiene `github.com/Nachodusan/tinyllm-ios`

- MLX Swift con Qwen2.5-0.5B en 4 bits sobre un iPhone 16: **108,85 tok/s en la primera generación y 66,32 sostenidos** en 150 corridas seguidas (61 % retenido). Los dos son correctos; sólo uno describe usar la cosa.
- *ProcessInfo.thermalState* llega tarde: cuando iOS reportó presión térmica por primera vez, el throughput ya había perdido un cuarto. Una app que sólo degrada al cambiar de estado ya le entregó ese cuarto al usuario.
- Una curva que baja tiene una segunda explicación idéntica por dentro —estado acumulado en el proceso—, así que la suite enfría tres minutos y repite el lote: **recuperó el 99 % de la ráfaga con la memoria plana** en las 300 generaciones. Eso convierte “térmico” en un hallazgo y no en una suposición.
- La primera medición fueron 12 corridas y decía “retenido 86 %, sin throttling”: aritméticamente correcta, equivocada por 39 %. Quedó guardada como evidencia y el informe ahora se advierte solo. Swift, xcodegen, devicectl; 19 tests.

PROYECTOS · PRODUCTO

Bitácora — diario hablado con IA para acompañar la terapia · iOS, Android y backend, de punta a punta

- Apps nativas en SwiftUI y Jetpack Compose con transcripción de voz on-device: el audio no sale del teléfono. Offline-first, con cola de envíos que sincroniza sin duplicar.
- El psicólogo recibe un informe mensual donde cada dato está respaldado por la cita literal del paciente, que nunca ve el informe para no empujarlo al autodiagnóstico. Detección de riesgo en cliente y servidor.
- Memoria de largo plazo con pgvector: los resúmenes de cada día se vectorizan, se recuperan y se reordenan en cada turno.
- Fastify + PostgreSQL (jsonb, migraciones incrementales) en VPS Ubuntu con systemd, red privada Tailscale e instancia aislada por cliente, con tope mensual de gasto en IA y costo por llamada registrado. **268 tests automatizados** (151 backend, 96 iOS, 21 Android) y simulación de un mes completo de uso.

Tutor IA — facilitador de aprendizaje sobre material oficial del Mineduc

- RAG de doble corpus tipado: el texto del estudiante como fuente citable y la guía docente como pedagogía anclada al objetivo de aprendizaje.
- Retrieval híbrido (denso + léxico + RRF) con reranker como gate de abstención, verificación NLI direccional por oración y detección de premisa falsa contra alucinaciones, corriendo en el chat real.
- Harness propio con golden set anclado a fragmentos reales: **recall@5 de 88 % sobre ~38.000 fragmentos en 35 asignaturas**. Un experimento midió el techo del modelo en matemática y la materia salió del alcance.

IA Rioplatense — chat por voz con LLM local, sin APIs cloud `github.com/Nachodusan/ia-rioplatense`

- Llama 3.1 8B vía MLX, 100 % local en Apple Silicon, con RAG BM25 sobre 319 ejemplos curados y un post-procesador léxico de 150+ reglas contextuales para garantizar voseo y vocabulario rioplatense.
- Pipeline con SSE streaming del LLM y cola TTS secuencial: texto progresivo en pantalla y voz arrancando en ~1 s.
- **Descarté tres iteraciones de fine-tuning LoRA** con las métricas A/B a la vista y documenté el diseño que las reemplazó.

Antes: plataforma de comercio electrónico con panel administrativo, sistema de registro masivo de datos (web y móvil) y plataforma IoT para monitoreo de sensores con dashboard en tiempo real — React, Node.js, PostgreSQL, MongoDB, Arduino, Kotlin y Flutter.

TECNOLOGÍAS

EVALUACIÓN	Golden sets · verificación de trayectoria · repeats y flakiness · recall@k contra fuerza bruta · verificación NLI · gates de abstención · trazas con latencia, tokens y costo
IA / LLMS	LangGraph · tool calling · RAG híbrido (denso + léxico + RRF) · rerankers (BGE-M3, Qwen3) · MLX y MLX Swift · Llama 3.1 · salidas estructuradas · prompt engineering · Piper TTS
DATOS	PostgreSQL · pgvector · Qdrant · Chroma · SQLite · MongoDB · MySQL
LENGUAJES	Python · TypeScript / JavaScript · Swift · Kotlin · C/C++
BACKEND	FastAPI · Node.js · Fastify · REST APIs
FRONTEND	React · HTML5 · CSS3
MOBILE	SwiftUI · SwiftData · Jetpack Compose · Flutter
INFRA	Linux (Ubuntu) · systemd · Docker · Tailscale · Vultr · Git y GitHub · GitHub Actions

EDUCACIÓN

Desarrollador web — TripleTen LatAm (oct. 2025 – sept. 2026)

Ingeniería en Informática — Universidad de Buenos Aires (en curso)

Curso de JavaScript — CoderHouse (2022)

Bachiller en Medicina — Universidad de Buenos Aires (2020 – 2022)

IDIOMAS

Español — nativo, rioplatense y Chile

Inglés — intermedio